Formalisation du modèle réellement implémenté `eLLM_v1`

Correspondance opérationnelle avec `DDD_Base`

Spécification technique, théorique et expérimentale

Version 1.0 – 21 octobre 2026

Résumé

Ce document formalise l'implémentation v1 comme elle est écrite dans le dépôt, en séparant strictement les primitives de `DDD_Base`, les conséquences mathématiques sous hypothèses, et les choix d'ingénierie appliqués au langage. Contrairement à eLLM_v0, les empreintes v1 ne sont plus des suffixes de tokens placés à côté d'un prédicteur indépendant : elles sont obtenues par des sondes apprises sur les états cachés contextualisés d'un modèle causal préentraîné, et une correction issue des sondes est ajoutée aux logits et supervisée à plusieurs résolutions. Le checkpoint préentraîné est choisi explicitement et son absence est une erreur bloquante, sans repli vers un n-gramme ou des poids aléatoires. Le téléchargement du checkpoint officiel n'a pas pu être réalisé dans l'environnement de construction, de sorte que le contrat logiciel et les tests de composants sont vérifiés, mais le fine-tuning sur les poids réels reste à exécuter sur une machine connectée.

Table des matières

1 Statut, sources et règle de fidélité **2**

2 Noyau observationnel repris de DDD_Base **2**
2.1 Objets, contexte, sondes et fonctionnel . . . 2
2.2 Échelles emboîtées, empreintes, fibres et quotient . . . 2
2.3 Fermeture et factorisation observationnelles . . . 3
2.4 Capacité effective, dimension effective et géométrie . . . 3
2.5 Seuils, frontière et distance dérivée . . . 3
2.6 Transitions compatibles, contraintes et imagination . . . 4
2.7 Expérience discriminante sur des fibres non triviales (P06) . . . 4

3 Architecture causale effectivement codée **4**
3.1 Substrat préentraîné et contrat de chargement . . . 4
3.2 Import local et identité du checkpoint . . . 5
3.3 Gate exécutable de validation locale P07 . . . 5
3.4 Sondes apprises multi-résolution . . . 5
3.5 Effet causal sur la prédiction . . . 6

4 Apprentissage, validation et test réservé **6**
4.1 Format d'exemples et segmentation . . . 6
4.2 Fonction de perte . . . 6
4.3 Normalisation par tokens et accumulation exacte . . . 6
4.4 Sélection et protocole d'évaluation . . . 7

5 Chat, historique, recherche et actions **7**

6 Traçabilité de l'implémentation **7**

7 État expérimental et limites observées 8

8 Commandes d'exécution 8

9 Conclusion et statut du modèle 9

1 Statut, sources et règle de fidélité

La source théorique de référence est `DDD_Base_theorie_des_empreintes_complete_order_5.tex`, conservée sans modification sous `Dmodel_eLLM_v1/reference/`. L'archive source `resources_0.zip` et les datasets sont conservés à l'identique. Les documents v0 sont seulement des artefacts d'audit et ne remplacent pas `DDD_Base`.

Trois statuts sont utilisés. **[BASE]** : notion explicitement contenue dans DDD_Base. **[DÉRIVÉ]** : résultat mathématique valide sous les hypothèses indiquées. **[APPLICATION]** : mécanisme logiciel qui instancie une partie de la théorie, sans prétendre être une primitive théorique. Les résultats sur un domaine fini demeurent finis ; ils ne deviennent pas des théorèmes universels.

2 Noyau observationnel repris de DDD_Base

2.1 Objets, contexte, sondes et fonctionnel

[BASE] Soient $\mathcal{P}$ l'ensemble des contextes d'observation, $\mathcal{X}$ le domaine des objets et $\mathcal{F}$ une famille de sondes. Le fonctionnel d'observation est

$$\Phi : \mathcal{P} \times \mathcal{X} \times \mathcal{F} \longrightarrow \mathbb{R}, \qquad (P, x, f) \mapsto \Phi(P, x, f).$$

Le contexte intervient dans le protocole : deux observations d'un même objet n'ont pas à être identiques si leur tâche, leur position, la langue, l'historique ou l'état interne observé diffèrent.

[APPLICATION] Dans eLLM_v1, un objet est principalement un préfixe linguistique tokenisé $x = (t_1, \ldots, t_n)$. Le contexte observé P consigne notamment la tâche, la langue si connue, l'identifiant de conversation, la position de token, le niveau de résolution et la provenance. Le substrat causal transforme les tokens en états contextualisés $h_j^{(\ell)}$. Les sondes apprises lisent l'état caché final pour chaque position. Cet observateur ne prétend pas mesurer une « signification absolue ».

2.2 Échelles emboîtées, empreintes, fibres et quotient

[BASE] Une échelle est une famille emboîtée $\mathcal{F}_1 \subseteq \cdots \subseteq \mathcal{F}_K$ accompagnée d'un ordre et, en version générale, d'une intensité ι. L'empreinte est

$$I^P_{\Phi,k}(x) = \big(\Phi(P, x, f)\big)_{f \in \mathcal{F}_k}.$$

L'indiscernabilité exacte est $x \sim_{P,k} y \iff I_{P,k}(x) = I_{P,k}(y)$. Elle induit la fibre $[x]_{P,k}$, le quotient $\mathcal{X}/\sim_{P,k}$ et l'image observationnelle $\mathcal{L}_{P,k} = I_{P,k}(\mathcal{X})$.

[APPLICATION] L'adaptateur de sondes est Obs_ω. Pour chaque niveau k, il produit un groupe de réponses scalaires $z_k(x) \in \mathbb{R}^p$, où $p =$ `probes_per_level`. Le niveau k utilise le préfixe de groupes

$$Z_k(x) = [z_1(x) \| z_2(x) \| \cdots \| z_k(x)] \in \mathbb{R}^{kp}.$$

Les signatures exactes de l'implémentation sont *exactes relativement au protocole quantifié déclaré* :

$$q_\delta(u) = \begin{cases} \left\lfloor \frac{u}{\delta} + \frac{1}{2} \right\rfloor, & u \geq 0, \\ \left\lceil \frac{u}{\delta} - \frac{1}{2} \right\rceil, & u < 0, \end{cases} \qquad \widehat{I}_k(x) = \big(q_\delta(z_1(x)), \ldots, q_\delta(z_k(x))\big).$$

Cette quantification est un choix d'implémentation. Elle n'affirme pas l'égalité des réponses continues lorsque deux signatures quantifiées coïncident.

Proposition 2.1 (Raffinement inter-échelles)**.** *Si $\mathcal{F}_k \subseteq \mathcal{F}_{k+1}$, alors $x \sim_{P,k+1} y \Rightarrow x \sim_{P,k} y$. Dans la version calculatoire, le préfixe exact de $\widehat{I}_{k+1}$ de largeur kp est $\widehat{I}_k$ par construction.*

Démonstration. L'égalité de deux vecteurs concaténés entraîne l'égalité de toutes les coordonnées de leur préfixe commun. La propriété est donc algébrique et n'est pas un résultat empirique sur le langage. □

2.3 Fermeture et factorisation observationnelles

[BASE] Pour une famille source G, la fermeture est

$$\mathrm{Cl}_{P,k}(G) = \{f : \exists \psi_f \text{ telle que } \Phi(P, x, f) = \psi_f(I^P_{\Phi,G}(x)) \; \forall x \in \mathcal{X}\}.$$

Elle est extensive, monotone et idempotente dans le cadre théorique déclaré. Le théorème de factorisation dit que $T : \mathcal{X} \to Y$ descend sur l'image de l'empreinte si et seulement si elle est constante sur ses fibres.

[APPLICATION] `finite_closure` et `factorization_report` testent cette propriété sur une liste finie d'objets fournie. Pour chaque signature source, le code recherche des réponses cibles contradictoires. Absence de conflit signifie uniquement « facteur sur cet échantillon et cette quantification » ; elle ne prouve pas une factorisation globale. Une partition en singletons rend le test trivial, donc les rapports doivent indiquer le nombre de fibres et les fibres non triviales.

2.4 Capacité effective, dimension effective et géométrie

[BASE] L'équivalence observationnelle des sondes compare leurs fonctions réponses sur le domaine. La capacité effective est $C_{\mathrm{eff},P}(k) = |\mathcal{F}^\sharp_{k,P}|$. La dimension effective est

$$d_{\mathrm{eff},P}(G) = \dim \mathrm{Aff}\big(I^P_{\Phi,G}(\mathcal{X})\big).$$

[APPLICATION] `effective_probe_capacity` compte les fonctions-réponses quantifiées distinctes sur le domaine déclaré ; `effective_affine_dimension` calcule le rang numérique de $I(X) - I(x_0)$. Le pas de quantification et la tolérance numérique font partie des métadonnées. Ces mesures sont des estimations sur les objets fournis et non des dimensions latentes universelles.

2.5 Seuils, frontière et distance dérivée

[BASE] Sur une échelle discrète finie, eLLM_v1 emploie le seuil opérationnel

$$\rho_K(x, y) = \max\{k \in \{0, \dots, K\} : \widehat{I}_k(x) = \widehat{I}_k(y)\}$$

et rapporte à part `boundary_status`, qui indique si l'indiscernabilité demeure au niveau maximal. Cette représentation discrète ne se substitue pas à la construction générale par $\sup \iota(\lambda)$ de DDD_Base, dont le statut de l'égalité critique dépend de la donnée de bord.

La distance dérivée sur les identifiants distincts est définie par

$$d_\rho(x, y) = \begin{cases} 0, & x = y, \\ e^{-\rho_K(x,y)}, & x \neq y. \end{cases}$$

La diagonale est donc nulle. Pour l'interprétation quotient, on définit séparément une pseudoultramétrique qui vaut zéro quand les signatures terminales sont égales, y compris pour deux identifiants distincts. Les deux constructions ne doivent pas être confondues.

2.6 Transitions compatibles, contraintes et imagination

[BASE] DDD_Base définit la compatibilité d'une transition par préservation des fibres, des trajectoires et des diagrammes de transport ; il traite également la fermeture exécutable, les objets contraintes, la provenance, l'imagination conservatrice, la révision, l'identité et les déficits. **[APPLICATION]** `transition_compatible` et `verify_transport_commutation` ne vérifient que le domaine fini fourni. Le contrôle de commutation vérifie maintenant deux conditions séparées : l'état bas fourni doit être exactement la restriction de l'état haut, puis les deux chemins du diagramme doivent produire des résultats égaux. `threshold_pair` exige un niveau zéro explicitement vide et des signatures réellement emboîtées ; un protocole mal formé est rejeté au lieu de recevoir un seuil trompeur. `PublicConstraint` conserve identifiant, type, valeur, source et statut public. `sample_imaginations` produit des branches candidates à graines explicites ; le contrôle de contraintes est déterministe, mais ne certifie ni le vrai ni une action externe. Les déficits $D_{obs}, D_{cap}, D_{res}, D_{search}, D_{transfer}, D_{constraint}$ sont des proxys opérationnels, non des diagnostics d'une incapacité intrinsèque.

2.7 Expérience discriminante sur des fibres non triviales (P06)

[EMPIRIQUE – DOMAINE FINI SYNTHÉTIQUE] Le script `Amodel_eLLM_v1/experiments/run_p06_` applique cinq fonctions-réponses à cinq objets explicitement déclarés, avec un pas de quantification de 1. Les nombres de classes par résolution sont $1, 2, 3, 3, 5, 5$. Il reste donc des fibres non triviales aux niveaux 1–3 ; le niveau 4 ajoute une séparation effective et le niveau 5 n'en ajoute pas dans cet échantillon.

La capacité effective mesurée est $C_{\text{eff}} = 4$ pour cinq sondes brutes, car deux sondes produisent la même fonction-réponse quantifiée sur le domaine. La dimension affine empirique est 4. La famille de niveau 1 ne permet pas la factorisation d'une opération cible ; après ajout d'une seconde sonde (niveau 2), cette opération est constante sur les fibres et factorise sur ce domaine. Une autre opération reste non factorisante au niveau 3, ce qui montre que le test n'est pas limité à un succès de factorisation trivial. Les 125 triplets satisfont la loi du minimum double sans violation.

Ces chiffres valident les algorithmes sur cette matrice finie, pas la sémantique d'une sonde réelle ni le comportement du checkpoint SmolLM2. L'expérience est conçue pour falsifier le calcul opérationnel : elle échouerait si les partitions ne se raffinaient pas, si la fermeture ne changeait pas avec la famille, si une capacité était comptée à partir des simples coordonnées au lieu des fonctions-réponses, ou si le transport ne détectait pas un diagramme non commutatif.

3 Architecture causale effectivement codée

3.1 Substrat préentraîné et contrat de chargement

Soit B_{θ_0} un véritable modèle causal préentraîné disponible localement, avec paramètres initiaux θ_0, tokenizer τ, embeddings de sortie W_{out} et représentation cachée $h_j(x)$. Le checkpoint par défaut déclaré est `HuggingFaceTB/SmolLM2-135M-Instruct`, épinglé à la révision immuable `ee72e5415c002e8fc0566a191a07c0b2b708875a`. Le chargeur inspecte la configuration et les fragments de poids, vérifie la présence de chaque shard référencé par un index, rejette les index invalides/chemins de shards dangereux, hache les fichiers, écrit un manifeste et refuse de charger si les poids sont absents/incomplets. Aucune branche de fallback vers n-gramme ou des poids aléatoires n'existe.

La fiche et la page du fichier officielles déclarent un modèle causal instruction-tuned de 135M paramètres sous Apache-2.0 ; la révision épinglée et le SHA-256 attendu de `model.safetensors` sont documentés à https://huggingface.co/HuggingFaceTB/SmolLM2-135M-Instruct/blob/ee72e5415c002e8fc0566a1 `model.safetensors` (SHA-256 `5af571cbf074e6d21a03528d2330792e532ca608f24ac70a143f6b369968ab8c`). Le dépôt v1 n'embarque pas ces poids car les tentatives vers le dépôt officiel, y compris l'URL de

cette révision précise, ont échoué dans l'environnement de construction. Cela bloque l'exécution d'un fine-tuning réel dans cet environnement, pas l'existence du code de chargement reproductible.

3.2 Import local et identité du checkpoint

Soit p le chemin absolu fourni par l'utilisateur, $M(p)$ l'ensemble des artefacts du checkpoint et H la fonction SHA-256. L'import ne confond pas la présence d'un fichier avec son authenticité :

$$\text{ImportLocal}(p) = \begin{cases} \text{Refus}, & \text{si configuration, poids ou shards requis sont absents/incohé} \\ \text{Refus}, & \text{si le checkpoint officiel épinglé ne respecte pas le hash atten} \\ \text{PublicationAtomique}(M(p)), & \text{si les contrôles sont satisfaits.} \end{cases}$$

Le contenu est d'abord copié dans un répertoire de staging ; un manifeste SHA-256 est produit puis vérifié avant publication. Un remplacement non identique exige une confirmation explicite et archive l'ancien état. Les liens symboliques sont refusés sauf les liens internes d'un snapshot Hugging Face dont les cibles sont dans le répertoire `blobs/` du même dépôt ; les fichiers copiés sont matérialisés. L'API exposée est `/api/model/import-local`, raccordée à la vue Réglages. Les tests couvrent ces branches avec checkpoints de fixture. La machine de construction ne voit pas le chemin du PC de l'utilisateur : aucune inférence ni aucun fine-tuning avec son exemplaire local ne sont donc revendiqués dans ce rapport. Le validateur local et ses cinq tests sont livrés, mais l'exécution sur le checkpoint utilisateur reste le gate P07 en attente.

3.3 Gate exécutable de validation locale P07

Soit p un répertoire local de checkpoint et D un dataset fourni. La procédure de validation ne considère pas l'existence de fichiers comme une preuve d'authenticité : elle vérifie séparément la structure de p, son manifeste de provenance, les hashes de fichier, l'identifiant/révision attendus et le chargement effectif par le moteur causal. Soit $L(p)$ le fait que les vrais poids ont été chargés et $G(p, q)$ le fait qu'une génération a été exécutée sur une requête q. Le statut d'inférence réelle ne peut être positif que si

$$\text{PASS}_{\text{infer}}(p, q) \iff \text{DepsOK} \wedge \text{StructureOK}(p) \wedge \text{ProvenanceOK}(p) \wedge L(p) \wedge G(p, q).$$

Pour le fine-tuning, soit $T(p, D, \eta)$ l'exécution avec paramètres η, B l'identité SHA-256 inchangée du dataset, V la sélection par validation, C^* le meilleur checkpoint publié et E_{test} l'évaluation réservée. On exige

$$\text{PASS}_{\text{train}} \iff \text{PASS}_{\text{infer}} \wedge B \wedge T \wedge V \wedge \text{Verify}(C^*) \wedge E_{test},$$

où E_{test} ne participe jamais au choix de C^*. Ces prédicats sont des contrats logiciels falsifiables ; ils ne sont pas des théorèmes sur la qualité du modèle ni sur la théorie des empreintes.

Le script `tools/validate_local_checkpoint_eLLM_v1.py` implémente ce gate sur la machine qui contient effectivement les poids. Ses rapports distinguent explicitement `official_checkpoint_loaded`, `real_model_inference_executed`, `real_model_finetuning_executed` et `heldout_evaluation_executed`. Le rapport conserve deux prédicats distincts : `model_checkpoint_loaded` atteste le chargement d'un checkpoint local validé, tandis que `official_checkpoint_loaded` n'est vrai que si le poids principal correspond au SHA-256 officiel épinglé. Une identité personnalisée peut être chargée sans être qualifiée d'officielle. Une exécution utilisant une fixture, un simple inventaire ou un test de dépendance manquante ne peut donc pas être interprétée comme une validation réelle du modèle.

3.4 Sondes apprises multi-résolution

Pour la position j, le groupe de niveau k est un petit réseau différentiable :

$$z_{j,k} = \tanh(W_{k,2} \tanh(\text{LN}(W_{k,1} h_j + b_{k,1})) + b_{k,2}) \in \mathbb{R}^p.$$

La largeur intermédiaire est le paramètre $d = \texttt{probe_dim}$, tandis que $p = \texttt{probes_per_level}$ est le nombre de réponses scalaires par niveau. Le niveau k agrège les premiers groupes et possède sa propre projection de retour vers la dimension cachée.

3.5 Effet causal sur la prédiction

Les logits de base sont $\ell_j^0 = W_{out} h_j + b$. Pour chaque résolution k, la correction est

$$\Delta\ell_{j,k} = W_{out}\, T_k(Z_{j,k}), \qquad \ell_{j,k} = \ell_j^0 + \tanh(g_k)\Delta\ell_{j,k},$$

où $T_k : \mathbb{R}^{kp} \to \mathbb{R}^{d_h}$ est un transport appris. La distribution prédictive est

$$p_{\theta,k}(t_{j+1} \mid t_{\le j}) = \mathrm{softmax}(\ell_{j,k}).$$

Ainsi, lorsque l'adaptateur est activé, il entre dans le chemin des logits ; il n'est pas seulement un tableau de diagnostics. Lorsqu'aucun adaptateur entraîné n'est présent, le modèle utilise les logits du checkpoint causal de base, et les diagnostics neuronaux qui dépendent des sondes sont désactivés. Un adaptateur aléatoire n'est jamais chargé comme s'il était entraîné.

4 Apprentissage, validation et test réservé

4.1 Format d'exemples et segmentation

Le pipeline importe JSONL, JSON, CSV, TSV, TXT, Markdown, ZIP et un sous-ensemble Parquet déclaré. Le fichier Parquet fourni a été décodé par `parquet_lite` sans PyArrow (7 473 lignes, colonnes UTF-8 `question`/`answer`, 8 groupes, compression Snappy) ; d'autres encodages/codecs Parquet peuvent nécessiter `pyarrow`. Il calcule un SHA-256 source, un numéro de ligne et un hash pour chaque ligne rejetée, détecte les champs, puis crée les exemples de conversation, prompt-cible, étiquette ou texte causal selon un mapping utilisateur explicite. Un enregistrement invalide est rapporté ; les sources brutes ne sont jamais réécrites.

Les empreintes de contenu exactes sont regroupées avant le découpage. Les fractions $r_{tr}, r_{val}, r_{test}$ doivent être positives et sommer à un. Le rapport garde les hashes par split et contrôle les intersections. Cette séparation réduit la fuite par doublons exacts mais n'élimine pas nécessairement les paraphrases ni les fuites sémantiques.

4.2 Fonction de perte

Pour la résolution maximale K :

$$\mathcal{L} = \mathcal{L}_{LM,K} + \lambda_{MR} \frac{1}{K-1} \sum_{k=1}^{K-1} \mathcal{L}_{LM,k} + \lambda_{div} \mathcal{L}_{div}.$$

$\mathcal{L}_{LM,k}$ est la cross-entropie causale calculée seulement sur les tokens cibles (les tokens de contexte sont masqués par -100). Le terme multi-résolution supervise les corrections à des échelles inférieures ; le terme diversité pénalise la corrélation des réponses de sonde sur le batch, sans être présenté comme une loss sémantique. Le fine-tuning peut être LoRA ou complet. Le mode LoRA ajuste les modules cibles compatibles avec le checkpoint et l'adaptateur de sondes ; le mode complet autorise le fine-tuning des paramètres du backbone selon les capacités du matériel.

4.3 Normalisation par tokens et accumulation exacte

Le code accumule des groupes de micro-lots G_q. Soit N_q le nombre exact de labels cibles non masqués après décalage causal dans le groupe :

$$N_q = \sum_{b \in G_q} \sum_{t} \mathbf{1}\{y_{bt} \neq -100\}.$$

La contribution LM au groupe est normalisée par N_q, et non par le nombre de micro-lots :

$$\mathcal{L}_{LM,k}^{(G_q)} = \frac{1}{N_q} \sum_{b \in G_q} \sum_{t : y_{bt} \neq -100} - \log p_{\theta,k}(y_{bt} \mid x_{b,\leq t}).$$

Le terme diversité est moyenné sur les micro-lots du groupe, puis $\mathcal{L}^{(G_q)}$ prend la forme de l'équation précédente avec ces termes de groupe. La réduction de cross-entropie est calculée en FP32 même lorsque les logits de backbone sont en FP16/BF16 ; la conversion reste différentiable. Ainsi, un dernier groupe d'accumulation incomplet n'est pas divisé par le nombre nominal de micro-lots et les lots courts ne reçoivent pas artificiellement le même poids token-wise que les lots longs. Ces règles sont testées numériquement en comparant pertes et gradients à ceux d'un lot concaténé.

Les états de reprise enregistrent les paramètres entraînables (backbone complet en mode full, ou paramètres LoRA et sondes en mode LoRA), buffers, état optimiseur, RNG Python/Torch/CUDA et générateur de shuffle. Ils sont liés au SHA-256 de l'arbre du checkpoint de base, au hash du dataset et à un hash de compatibilité de configuration. Une reprise est refusée si un de ces identifiants ne correspond plus. Les poids sélectionnés par NLL de validation sont rechargés avant l'évaluation du test réservé ; `eval_every_epochs` déclenche les évaluations à l'intervalle configuré et toujours à la dernière époque.

4.4 Sélection et protocole d'évaluation

La NLL/perplexité de validation choisit le meilleur epoch. Les métriques de test réservé ne sont pas calculées pendant cette sélection. Après sélection, le code évalue (i) le checkpoint sélectionné avec l'adaptateur, (ii) les mêmes poids sélectionnés avec contribution empreinte désactivée, et (iii) un backbone préentraîné rechargé pour la baseline. Les métriques consignées comprennent NLL, perplexité, nombre de tokens scorés, hashes dataset/splits, configuration, historique d'epochs et statut de l'accès au test. Un run interrompu conserve son état de reprise et n'est pas appelé « terminé ».

5 Chat, historique, recherche et actions

Les messages sont persistés dans SQLite sous `sessions/history.sqlite3`, avec timestamps, identifiant de modèle et métadonnées de génération. Recherche et imagination réutilisent le modèle causal courant ; les graines sont enregistrées. La révision génère un candidat à partir d'un candidat fourni et de contraintes explicitement publiques, puis vérifie ces contraintes. Les candidats imaginés ne sont pas présentés comme faits vérifiés, et les opérations externes ne sont pas exécutées par ces modules.

L'interface Web inclut les vues Chat, Entraînement, Empreintes, Expériences et Réglages. Elle permet import multi-format, aperçu, mapping des champs, configuration, lancement/annulation et observation des jobs, export/renommage/suppression de chats, comparaison d'empreintes, recherche et imagination. Le serveur écoute sur loopback par défaut. Les utilisateurs qui exposent le service sur un réseau doivent ajouter une politique d'authentification ; l'interface locale n'implémente pas une authentification multi-utilisateur.

6 Traçabilité de l'implémentation

Concept / statut	**Code principal**	**Preuve et portée**
$\Phi, P, \mathcal{X}, \mathcal{F}$ [BASE/APPLICATION]	`theory.py`, `model.py`	Contexte et réponses des sondes sur les états cachés ; tests du wrapper causal jouet.

Concept / statut	Code principal	Preuve et portée
Familles emboîtées, signatures [DÉRIVÉ]	`fingerprint_adapter.py`	Préfixe exact par concaténation ; égalité exacte après quantification déclarée.
Fibres / quotient / raffinement [BASE/DÉRIVÉ]	`theory.py`	Partitions et raffinement sur objets finis ; tests unitaires.
Fermeture / factorisation [BASE]	`finite_closure`, `factorization_report`	Conflits recherchés sur domaine fourni ; aucune preuve universelle.
Seuil, bord, ultramétrique [BASE/DÉRIVÉ]	`threshold_pair`, `threshold_metric`	Diagonale nulle et bord explicite ; test.
Capacité / dimension effective [BASE/APPLICATION]	`effective_probe_capacity`, `effective_affine_dimension`	Comptage/rang de réponses observées sur domaine fini, seuils numériques déclarés.
Transport / contraintes / déficit [BASE/APPLICATION]	`theory.py`, `diagnostics.py`, `engine.py`	Test de compatibilité fini, provenance des contraintes, proxys déclarés.
Apprentissage fondé sur l'empreinte [APPLICATION]	`trainer.py`, `fingerprint_adapter.py`	Correction des logits et loss multi-résolution ; unité de test confirme effet non nul sur logits du modèle fixture.
Dataset et provenance [APPLICATION]	`dataio.py`	Hash du fichier ; lignes invalides ; mapping ; splits par hashes exacts ; datasets fournis.
Chat persistent [APPLICATION]	`history.py`, `server.py`	SQLite, recherche, export, renommage, suppression ; tests de persistance.

7 État expérimental et limites observées

Au 21 octobre 2026, la régression locale après ajout du harnais de validation P07 compte 103/103 tests réussis, et la compilation Python, l'aide CLI du runtime et la syntaxe JavaScript sont passées. Les tests de perte et de téléchargement intègrent des fixtures explicitement déclarées. Le test Chromium indépendant compte 13/13 contrôles réussis avec API simulée ; les routes d'import sont également vérifiées par les tests HTTP. Les tests P06 dédiés comptent 19/19 réussites ; l'expérience synthétique rapporte 125 triplets et zéro violation de la loi du minimum double. Les tests locaux peuvent vérifier le code des signatures, la factorisation finie, la perte différentiable sur un modèle fixture et les routes Web sans modèle téléchargé. Ils ne remplacent pas une exécution de fine-tuning sur le checkpoint officiel. Dans cet environnement, la tentative de télécharger même le manifeste `config.json` du dépôt a échoué. Par conséquent, aucun chiffre de perplexité ou gain de performance de eLLM_v1 sur le checkpoint réel n'est revendiqué ici. Le lancement, l'entraînement et l'évaluation avec les poids téléchargés sont une étape externe documentée, non une réussite présupposée.

Les constructions d'imagination, de recherche, de contraintes, de mémoire et de déficits restent des applications opérationnelles partielles des développements de DDD_Base. Le dépôt contient une table de correspondance plus détaillée dans `Amodel_eLLM_v1/source_audit/ddd_base_mapping_eLLM_v1.json`. L'implémentation ne prétend pas épuiser toutes les constructions d'un document théorique de 3 922 lignes.

8 Commandes d'exécution

```
python setup_eLLM_v1.py
```

```
python run_eLLM_v1.py status
python run_eLLM_v1.py download-model --model-id HuggingFaceTB/SmolLM2-135M
   -Instruct --revision ee72e5415c002e8fc0566a191a07c0b2b708875a
python run_eLLM_v1.py serve --host 127.0.0.1 --port 8765
python run_eLLM_v1.py train --dataset training_data/raw/dataset/
   Screenplay_dataset.jsonl
python run_eLLM_v1.py selftest -v
node --check web/app.js
```

9 Conclusion et statut du modèle

Le gate P06 de cohérence formelle, tests de composants, navigateur mocké, traçabilité et compilation est passé. P07 reste ouvert : l'environnement de construction ne voit pas le système de fichiers du PC utilisateur et n'a donc ni chargé ses poids SmolLM2 ni exécuté le fine-tuning/held-out sur ces poids. Les archives de continuité doivent être lues comme le moyen d'exécuter cette étape sur la machine qui contient effectivement le checkpoint, et non comme une preuve d'un entraînement réel déjà accompli.

eLLM_v1 apporte une différence architecturale vérifiable par rapport à la v0 : le prédicteur est un véritable backend causal préentraîné lorsqu'un checkpoint est disponible ; les sondes apprises transforment ses états cachés ; les signatures emboîtées sont utilisées pour décrire le quotient observationnel ; une correction issue des empreintes participe aux logits et à la perte. Le chargement du poids réel, le fine-tuning et l'évaluation linguistique effective restent non exécutés dans l'environnement de génération des livrables, car le téléchargement du checkpoint y a échoué. La version v1 est donc une implémentation structurée, testée au niveau des composants et livrée avec un chemin de lancement, mais son gate empirique avec poids réels reste explicitement ouvert. Aucun verdict sur la réussite ou l'échec ultime du modèle n'est programmé dans le code.